



**Професионална гимназия по високи технологии  
“Александър Степанович Попов”**

гр. София п.к.1309, бул. Сливница № 10, тел. 02 / 822 11 86, 828 50 10  
<https://aspopov.bg/>, e-mail: [info-2206410@edu.mon.bg](mailto:info-2206410@edu.mon.bg)

---

**ДИПЛОМЕН ПРОЕКТ**

**НА ТЕМА:**

**Създаване на електронно пиано чрез езика C#**

Изготвил: Митко Стоянов

Ръководител: инж. Боян Мак Михайлов

Професия код 481020 „Системен програмист”

Специалност код 4810201, „Системно програмиране“

Клас: 12Е

София,

2026 г.

# Съдържание:

|                                                                                                   |    |
|---------------------------------------------------------------------------------------------------|----|
| УВОД .....                                                                                        | 5  |
| ГЛАВА I. ТЕОРЕТИЧНО ПРОУЧВАНЕ .....                                                               | 6  |
| 1.1. Създаване на музика с помощта на компютър – исторически преглед и съвременни тенденции ..... | 6  |
| 1.2. Звук и MIDI технология .....                                                                 | 7  |
| 1.2.1. Какво е MIDI? .....                                                                        | 7  |
| 1.2.2. MIDI съобщения и канали.....                                                               | 7  |
| 1.2.3. MIDI контролери и системни съобщения .....                                                 | 8  |
| 1.2.4. Програмна смяна на тембър .....                                                            | 8  |
| 1.3. Езикът C# и платформата .NET .....                                                           | 9  |
| 1.3.1. Основни характеристики на C# .....                                                         | 9  |
| 1.3.2. Windows Forms за създаване на графичен интерфейс .....                                     | 11 |
| 1.4. Библиотеки за работа със звук в C#.....                                                      | 11 |
| 1.4.1. Sanford.Multimedia.Midi – MIDI комуникация .....                                           | 11 |
| 1.4.2. NAudio – запис и експорт на аудио .....                                                    | 12 |
| 1.4.3. Сравнение на алтернативни библиотеки .....                                                 | 12 |
| 1.5. Анализ на съществуващи софтуерни пиана.....                                                  | 12 |
| 1.6. Избор на технологии и обосновка .....                                                        | 13 |
| ГЛАВА II. ПРОЕКТИРАНЕ НА ПРИЛОЖЕНИЕТО.....                                                        | 14 |
| 2.1. Анализ на изискванията .....                                                                 | 14 |
| 2.1.1. Функционални изисквания.....                                                               | 14 |
| 2.1.2. Нефункционални изисквания.....                                                             | 14 |
| 2.2. Потребителски роли.....                                                                      | 15 |
| 2.3. Потребителски истории (user stories) .....                                                   | 15 |
| 2.4. Примерни сценарии за работа .....                                                            | 16 |
| 2.5. Проектиране на класовете .....                                                               | 17 |
| 2.5.1. Основни класове.....                                                                       | 17 |
| 2.5.2. Проектиране на записа и възпроизвеждането .....                                            | 17 |
| 2.5.3. Обработка на грешки и изключения.....                                                      | 18 |
| 2.6. Проектиране на потребителския интерфейс.....                                                 | 18 |
| 2.6.1. Прототип .....                                                                             | 18 |

|                                                                      |           |
|----------------------------------------------------------------------|-----------|
| 2.6.2. Динамично генериране на клавишите – математически модел ..... | 19        |
| <b>ГЛАВА III. РЕАЛИЗАЦИЯ НА ЕЛЕКТРОННОТО ПИАНО .....</b>             | <b>20</b> |
| 3.1. Създаване на проекта във Visual Studio.....                     | 20        |
| 3.2. Структура на приложението .....                                 | 20        |
| 3.3. Реализация на графичния интерфейс .....                         | 21        |
| 3.4. Детайлно описание на класовете .....                            | 21        |
| 3.4.1. Клас MainForm .....                                           | 21        |
| 3.4.2. Структура RecordedMidiEvent.....                              | 22        |
| 3.4.3. Клас OutputDevice .....                                       | 22        |
| 3.4.4. Клас ChannelMessage .....                                     | 22        |
| 3.5. Осъществяване на ключови функционалности .....                  | 23        |
| 3.5.1. Генериране на клавиатурата .....                              | 23        |
| 3.5.2. Обработка на събития от мишка и клавиатура .....              | 23        |
| 3.5.3. MIDI изход и смяна на инструменти .....                       | 24        |
| 3.5.4. Прецизен запис със Stopwatch.....                             | 24        |
| 3.5.5. Възпроизвеждане с точност до милисекунда .....                | 24        |
| 3.5.6. Експорт към WAV чрез loopback запис .....                     | 25        |
| 3.5.7. Асинхронно програмиране и управление на нишки.....            | 25        |
| 3.5.8. Управление на ресурсите и освобождаване .....                 | 25        |
| 3.6. Демонстрация на основните методи .....                          | 26        |
| 3.6.1. Метод GetMidiNote.....                                        | 26        |
| 3.6.2. Метод HighlightKey .....                                      | 26        |
| 3.7. Тестване и валидация .....                                      | 27        |
| 3.7.1. Методика на тестване.....                                     | 27        |
| 3.7.2. Резултати от тестовете .....                                  | 27        |
| 3.7.3. Анализ на производителността .....                            | 28        |
| 3.7.4. Оптимизации и бъдещи подобрения .....                         | 28        |
| 3.8. Ръководство за потребителя.....                                 | 29        |
| <b>ЗАКЛЮЧЕНИЕ .....</b>                                              | <b>31</b> |
| <b>ПРИЛОЖЕНИЯ .....</b>                                              | <b>33</b> |
| Приложение 1: пълен код на MainForm.....                             | 33        |
| Фиг. 11 - пълен код на MainForm .....                                | 46        |

|                                                                |           |
|----------------------------------------------------------------|-----------|
| <b>Приложение 2: пълен код на MainForm.Designer.cs .....</b>   | <b>47</b> |
| <b><i>Фиг. 12 - пълен код на MainForm.Designer.cs.....</i></b> | <b>55</b> |
| <b>ИЗПОЛЗВАНА ЛИТЕРАТУРА:.....</b>                             | <b>56</b> |

# УВОД

Дипломният проект представя разработката на софтуерно електронно пиано, реализирано с езика C# и технологията Windows Forms. Разработката е изпълнение на индивидуално задание, което поставя следните изисквания: виртуалната клавиатура да поддържа всички основни тонове и полутонове в поне една октава (в реализацията са предвидени три октави – от C4 до C6 – за по-голяма гъвкавост), свиренето да става както с мишка, така и с компютърна клавиатура, натиснатият клавиш да се визуализира чрез промяна на цвета, а тембърът на звука да може да се променя. За белите клавиши от компютърната клавиатура са определени бутоните A, S, D, F, G, H, J, а за черните – W, E, T, Y, U. Звукът се генерира чрез MIDI протокола, като се използва вграденият софтуерен синтезатор на Windows.

Актуалността на темата идва от факта, че все повече хора използват компютрите за музика – за учене, композиране или забавление. Софтуерните пиана са достъпна алтернатива на физическите инструменти, но повечето съществуващи безплатни приложения имат ограничени възможности – рядко поддържат MIDI изход, а ако поддържат запис, той често е в собствен формат. Затова в този проект са добавени разширения: регулиране на силата на звука, показване на имената на нотите, запис на MIDI събития с точност до милисекунда, възпроизвеждане на записа и експорт към WAV файл. Така потребителят може не само да свири, но и да анализира изпълнението си, да запазва музикални идеи или да споделя готови записи. Допълнителна мотивация е учебният характер – създаването на такова приложение дава възможност да се приложат на практика знанията по обектно-ориентирано програмиране<sup>1</sup>, работа с графичен интерфейс, многопоточност<sup>2</sup> и аудио технологии.

Основните задачи, решавани в проекта, включват: проучване на MIDI технологията и библиотеките за C#, проектиране на класовата архитектура, реализация на динамично генериране на клавиатурата, осигуряване на две устройства за управление (мишка и клавиатура), MIDI изход с поддръжка на Program Change за смяна на тембър, запис със Stopwatch, прецизно възпроизвеждане с възможност за прекъсване и WAV експорт чрез loopback запис с NAudio. Дипломната работа е структурирана в три глави – теория, проектиране и реализация, последвани от заключение.

---

<sup>1</sup> Наков, С., & колектив. (2006). *Програмиране за .NET Framework*. София: БАРС. <http://www.devbg.org/dotnetbook/>

<sup>2</sup> Microsoft Docs. (2025). *Managed Threading in .NET*. Microsoft Learn. Достъпна на: <https://learn.microsoft.com/en-us/dotnet/standard/threading/>

# ГЛАВА I.

## ТЕОРЕТИЧНО ПРОУЧВАНЕ

### 1.1. Създаване на музика с помощта на компютър – исторически преглед и съвременни тенденции

Компютрите се използват за създаване на музика още от 50-те години на миналия век. Първите системи като CSIRAC (1951) и MUSIC I (1957) генерират звук изцяло цифрово. През 70-те и 80-те години се появяват цифрови синтезатори като Yamaha DX7, които революционизират музикалната индустрия. Когато персоналните компютри стават масови, софтуерните имитации на инструменти стават достъпни за всеки. В началото на 90-те звуковите карти с wavetable синтез (например Sound Blaster AWE32) позволяват реалистично звучене на MIDI инструменти. Днес виртуалните инструменти се използват не само от любители, но и от професионални музиканти в студийни условия.

През 1983 година се появява MIDI – стандарт, който позволява на електронни музикални устройства да комуникират помежду си. MIDI не пренася звук, а само инструкции като „натисни нота C5 със сила 100“. Благодарение на малкия обем данни, MIDI е изключително подходящ за софтуерни приложения. С помощта на MIDI днес работят почти всички виртуални инструменти, включително и електронните пиана, защото той осигурява бърза реакция и лесно управление на различни тембри. MIDI стандартът продължава да се развива – през 2016 г. е представен MIDI 2.0, който добавя двупосочна комуникация, по-висока резолюция на контролерите и по-голяма изразителност. В този проект се използва MIDI 1.0, тъй като той е широко поддържан от всички съвременни операционни системи.

Този проект използва MIDI изход към вградения синтезатор на Windows. По този начин се съчетава историческият опит на MIDI стандарта със съвременната среда .NET и C#, което прави приложението лесно за разширяване и преносимо между различни компютри. Съществуват и по-сложни подходи като генериране на звук директно чрез семпли, но за целите на този проект вграденият синтезатор е напълно достатъчен.

## 1.2. Звук и MIDI технология

### 1.2.1. Какво е MIDI?

MIDI представлява протокол за комуникация между музикални инструменти и компютри<sup>3</sup>. Предават се само команди – например „включи нота C5“, „смени инструмента на пиано“. Получаващото устройство (в случая софтуерният синтезатор) генерира звук според получените команди. Стандартът е въведен от MIDI Manufacturers Association и се използва в почти всички електронни музикални устройства от 80-те години насам. Едно MIDI съобщение се състои от статус байт (който определя типа на съобщението и MIDI канала) и един или два байта данни. Например Note On съобщение има статус байт 0x90 + канал, следван от номер на нота (0-127) и сила (velocity). Тази компактност позволява MIDI потоците да се предават дори през бавни серийни интерфейси.

### 1.2.2. MIDI съобщения и канали

MIDI има два основни вида съобщения – канални и системни. Каналните са за конкретен MIDI канал. Каналите са 16 (номерирани от 0 до 15), което позволява едновременно управление на 16 различни инструмента. Най-често използваните съобщения в този проект са:

- Note On – изпраща се при натискане на клавиш. Носи номер на нота (0-127) и сила на натиск (velocity, 0-127). Силата влияе на звука – по-голяма стойност дава по-силен и ярък тон.
- Note Off – изпраща се при отпускане на клавиш. Обикновено се изпраща със същия номер на нота и velocity 0. Някои синтезатори приемат Note On с velocity 0 като Note Off.
- Program Change – използва се за смяна на инструмент (тембър). С един байт данни (0-127) се избира нов инструмент от списъка General MIDI.

Всички съобщения в програмата се изпращат по MIDI канал 0, дефиниран от константата `midiChannel`. Това опростява логиката, тъй като не се налага управление на множество канали.

---

<sup>3</sup> MIDI Manufacturers Association. (1996). *The Complete MIDI 1.0 Detailed Specification*. [MIDI.org](https://midi.org/midi-1-0-detailed-specification). Достъпна на: <https://midi.org/midi-1-0-detailed-specification>

### 1.2.3. MIDI контролери и системни съобщения

Освен основните съобщения, MIDI поддържа и контрол промени (Control Change). Те се използват за управление на параметри като сила на звука, панорама, реверберация, сустейн педал и други. Например контролер номер 7 (Volume) променя силата на звука за даден канал. Контролер номер 64 (Sustain Pedal) се използва за удължаване на звука. В този проект вместо MIDI контролер за сила на звука се използва директно velocity в Note On съобщенията, което е по-просто и работи с всички синтезатори. Системни съобщения (System Exclusive, System Common, System Real-Time) не се използват, за да се запази простотата на кода.

### 1.2.4. Програмна смяна на тембър

Стандартът General MIDI дефинира 128 инструмента<sup>4</sup>. За този проект са важни следните номера:

- 0 – акустично пиано
- 19 – орган
- 56 – тромпет
- 73 – флейта
- 40 – цигулка

Потребителят избира инструмент от падащ списък, а програмата изпраща съответния номер като Program Change. Благодарение на General MIDI, тези номера са еднакви на всяка звукова карта, което гарантира, че избраният инструмент звучи правилно при различни компютри. General MIDI определя също така, че канал 10 е запазен за ударни инструменти, но тъй като се използва само канал 0, няма конфликт.

---

<sup>4</sup> General MIDI Association. General MIDI Level 1 Specification. [MIDI.org](https://midi.org). Достъпна на: <https://midi.org/general-midi>



## 1.3. Езикът C# и платформата .NET

### 1.3.1. Основни характеристики на C#

C# е обектно-ориентиран език, разработен от Microsoft като част от .NET платформата<sup>5</sup>. Той предлага автоматично почистване на паметта (garbage collector), строга проверка на типовете и богата стандартна библиотека. В проекта са използвани следните концепции:

- Обектно-ориентиран подход – класове, наследяване, полиморфизъм.
- Събитийно-ориентирано програмиране – подходящо за Windows Forms.
- Генерични колекции – List<T>, Dictionary<TKey,TValue>, HashSet<T>.
- Асинхронно програмиране – async/await, Task, CancellationToken.

В приложението са дефинирани следните полета (съгласно Form1.cs):

```
private List<Button> whiteKeys = new List<Button>();
private List<Button> blackKeys = new List<Button>();

// ----- MIDI -----
private OutputDevice midiOut;
private const int midiChannel = 0;           // MIDI канал

// ----- Октави -----
private int currentOctave = 5;
private readonly int[] allowedOctaves = { 4, 5, 6 };

// ----- Карти за клавишите от компютърната клавиатура -
// -----
private readonly Dictionary<Keys, int> whiteKeyMap = new Dictionary<Keys, int>()
{
    { Keys.A, 0 }, { Keys.S, 2 }, { Keys.D, 4 }, { Keys.F, 5 },
    { Keys.G, 7 }, { Keys.H, 9 }, { Keys.J, 11 }
};

private readonly Dictionary<Keys, int> blackKeyMap = new Dictionary<Keys, int>()
{
```

<sup>5</sup> Microsoft Docs. (2025). C# Programming Guide. Microsoft Learn. Достъпна на: <https://learn.microsoft.com/en-us/dotnet/csharp/>

```

        { Keys.W, 1 }, { Keys.E, 3 }, { Keys.T, 6 }, { Keys.Y, 8
    }, { Keys.U, 10 }
    };

    // Активно натиснати ноти (за да не се дублират)
    private HashSet<int> activeNotes = new HashSet<int>();

```

Фиг.1 - основни полета на класа

- List<Button> пази референции към всички бели и черни клавиши. Това позволява бързо прерисуване и промяна на визуалния им вид. Използването на динамичен списък улеснява добавянето и премахването на клавиши при смяна на октавата.
- Dictionary<Keys, int> свързва клавиш от компютърната клавиатура (например Keys.A) с полутонов индекс. Благодарение на речника, преобразуването става за константно време  $O(1)$ , което е критично за бърза реакция.
- HashSet<int> следи кои MIDI ноти в момента са активни. Това предотвратява изпращането на повторно Note On, докато нотата не бъде спряна с Note Off. Без тази проверка биха могли да възникнат нежелани ефекти като „залепване“ на нотите.

Записът на MIDI събития използва структура, а не клас, за да се намали натоварването на garbage collector-а<sup>6</sup>:

```

private struct RecordedMidiEvent
{
    public long TimeMs;
    public ChannelMessage Message;
}

```

Фиг.2 - структура за запис на MIDI събитие

Структурата (struct) се съхранява в стека или в непрекъснат блок памет в списъка, докато класът (class) би изисквал отделно заделяне в heap-а. При запис на дълги изпълнения (хиляди събития) това спестява памет и намалява фрагментацията. Използването на Stopwatch и CancellationToken гарантира, че възпроизвеждането на записа може да бъде спряно точно и навреме. Stopwatch използва високочестотния таймер на процесора (QueryPerformanceCounter), което дава точност до микросекунди.

<sup>6</sup> Albahari, J., & Albahari, B. (2017). *C# 7.0 in a Nutshell*. O'Reilly Media. Достъпна на: <https://www.oreilly.com/library/view/c-7-0/9781491987650/>

## 1.3.2. Windows Forms за създаване на графичен интерфейс

Windows Forms е технология за създаване на десктоп приложения<sup>7</sup>. Тя предоставя набор от контроли – бутони, панели, плъзгачи и други. Програмистът обработва събития (кликване, натискане на клавиш), за да реализира логиката на приложението. Windows Forms използва GDI+ за визуализация, което е достатъчно бързо за нуждите на виртуална клавиатура с около 30 клавиша. За разлика от WPF, Windows Forms има по-нисък праг на усвояване и е предпочитан в учебни проекти. За този проект Windows Forms е избрана, защото позволява динамично създаване на контроли – клавишите се генерират програмно, което улеснява смяната на октавата без промяна на кода.

## 1.4. Библиотеки за работа със звук в C#

.NET Framework не разполага с вградена MIDI поддръжка, поради което се налага използването на външни библиотеки. За този проект са избрани две библиотеки с отворен код.

### 1.4.1. Sanford.Multimedia.Midi – MIDI комуникация

Тази библиотека е разработена от Leslie Sanford<sup>8</sup>. Тя обвива winmm.dll (Windows Multimedia) и предоставя обектно-ориентиран интерфейс за MIDI вход и изход. Основните класове са:

- `OutputDevice` – представлява MIDI изход. Създава се с `new OutputDevice(0)`, като 0 съответства на Microsoft GS Wavetable Synth. Може да се изброят наличните устройства чрез `OutputDevice.DeviceCount`.
- `ChannelMessage` – капсулира MIDI съобщение за даден канал. Позволява лесно създаване на Note On, Note Off, Program Change и други.
- Методът `Send()` изпраща съобщението асинхронно. Библиотеката автоматично се грижи за опашката и избягва забавяния.

Библиотеката е стабилна, лека и няма външни зависимости. Тя работи директно с вградения синтезатор на Windows без допълнителни настройки. Алтернатива би била директно използване на `P/Invoke` към `winmm.dll`, но това би увеличило сложността и риска от грешки.

---

<sup>7</sup> Microsoft Docs. (2025). *Windows Forms Documentation*. Microsoft Learn. Достъпна на: <https://learn.microsoft.com/en-gb/dotnet/desktop/winforms/>

<sup>8</sup> Sanford, L. *Sanford.Multimedia.Midi* [GitHub repository]. GitHub. Достъпна на: <https://github.com/tebjan/Sanford.Multimedia.Midi>

## 1.4.2. NAudio – запис и експорт на аудио

NAudio е мощен звуков пакет, създаден от Mark Heath<sup>9</sup>. Той поддържа възпроизвеждане и запис на аудио чрез различни API – WaveOut, DirectSound, ASIO, WASAPI. За този проект са използвани следните компоненти:

- WasapiLoopbackCapture – записва аудиото, което звуковата карта възпроизвежда (loopback). Това позволява директно записване на реалния звук от MIDI синтезатора, без да се използват физически кабели или микрофони. WASAPI (Windows Audio Session API) е съвременният аудио интерфейс на Windows, който осигурява нисък latency и високо качество.
- WaveFileWriter – записва аудио данните във WAV файл. WAV е несвит формат, поддържан от всички аудио плеъри и редактори.
- WaveFormat – описва аудио формата (честота, битова дълбочина, брой канали). Loopback capture обикновено използва 32-битов float, 44100 Hz или 48000 Hz, стерео.

Експортът се извършва по следния начин: стартира се loopback capture, пуска се възпроизвеждане на записаните MIDI събития, аудиото се прихваща и записва. След приключване се изчакват 2 секунди, за да затихнат последните ноти, и записът спира. Това е елегантно решение, което не изисква допълнителен хардуер. Недостатък е, че записва всички звуци от системата, затова по време на експорт трябва да се изключат други аудио източници.

## 1.4.3. Сравнение на алтернативни библиотеки

Съществуват и други библиотеки за работа със звук в C#: DirectX/DirectSound (ниско ниво, сложна за използване), OpenTK (изисква OpenGL контекст), CSCore (по-малко популярна и с оскъдна документация). NAudio е избрана поради своята зрялост, обширна документация и активна общност. Sanford.Multimedia.Midi няма пряк конкурент в екосистемата на .NET – алтернативата Melanchall.DryWetMidi е по-тежка и с повече зависимости. Поради тези причини изборът на двете библиотеки е напълно оправдан.

## 1.5. Анализ на съществуващи софтуерни пиана

Направен е кратък преглед на няколко безплатни виртуални пиана, достъпни в интернет. Констатирано е, че повечето от тях не поддържат MIDI изход или записът е в собствен формат. Ето конкретни примери:

---

<sup>9</sup> Heath, M. NAudio [GitHub repository]. GitHub. Достъпна на: <https://github.com/naudio/NAudio>

- Online Piano (уеб базирани) – работят само в браузър, имат висока латентност (често над 50 ms) и не предлагат запис или експорт.
- SimplePiano (open source) – поддържа MIDI изход, но липсва функция за запис и визуализация на компютърна клавиатура.
- Virtual Piano (мобилни приложения) – често са ограничени до една октава и съдържат реклами.

В сравнение с тях, настоящото приложение предлага запис на MIDI събития с точно време, възпроизвеждане с възможност за прекъсване и експорт към WAV. То поддържа три октави и може лесно да бъде разширено до повече. Тези характеристики го правят значително по-полезно за музиканти и любители.

## 1.6. Избор на технологии и обосновка

Избраните технологии са обобщени в следния списък:

- C# – изучава се в училище, добре работи с Windows Forms, има богата стандартна библиотека.
- Windows Forms – позволява бърза разработка и динамично създаване на контроли.
- Sanford.Multimedia.Midi – стабилна MIDI библиотека с отворен код.
- NAudio – поддържа loopback capture, необходим за WAV експорт.
- Visual Studio 2022 – съвременна среда за разработка с дебъгер и дизайнер.
- .NET Framework 4.7.2 – работи на Windows 7, 8, 10 и 11 без допълнителни инсталации.

Този набор от технологии осигурява бърза работа, стабилност и възможност за бъдещо разширяване (например добавяне на MIDI вход чрез InputDevice от Sanford.Multimedia.Midi). Освен това, всички библиотеки са с лиценз MIT, което позволява свободно използване и разпространение.

## **ГЛАВА II.**

# **ПРОЕКТИРАНЕ НА ПРИЛОЖЕНИЕТО**

### **2.1. Анализ на изискванията**

#### **2.1.1. Функционални изисквания**

Съгласно индивидуалното задание, програмата трябва да изпълнява следните функции:

- Възпроизвежда звук при натискане на виртуален клавиш с мишката или с компютърната клавиатура.
- Поддържа всички тонове и полутонове в поне една октава (в реализацията са реализирани три октави – C4, C5, C6).
- Променя цвета на натиснатия клавиш (светлосин за белите, тъмносин за черните).
- Позволява смяна на тембър – пиано, орган, тромпет, флейта, цигулка.

Допълнителни функции, добавени по желание и надхвърлящи заданието:

- Плъзгач за регулиране на силата на звука (velocity).
- Показване на имената на нотите върху клавишите.
- Запис на изпълнение (MIDI събития с точност до милисекунда).
- Спиране на записа.
- Възпроизвеждане на записа с възможност за прекъсване.
- Експорт на записа към WAV файл чрез loopback запис.
- Автоматично преоразмеряване на клавишите при промяна на размера на прозореца.
- Подсказки при задържане на мишката върху бутоните.

#### **2.1.2. Нефункционални изисквания**

- Бързина – забавянето между натискане на клавиш и чуване на звук да бъде под 10 ms. Възпроизвеждането да е точно до милисекунда.
- Преносимост – приложението да работи на Windows 7, 8, 10 и 11 с .NET Framework 4.7.2.

- Стабилност – да няма грешки при бързо натискане на много клавиши, при едновременен запис и възпроизвеждане, както и при затваряне на програмата по време на активни операции. За целта се използва `HashSet<int>` за предотвратяване на дублирани MIDI съобщения.
- Интерфейс – да бъде лесен за използване, с контрастни цветове и подсказки.

Тестовите потвърждават, че забавянето е под 5 ms, а точността на възпроизвеждане е до 2 ms.

## 2.2. Потребителски роли

В приложението съществува само една роля – Потребител. Не се изисква регистрация, вход или администраторски права. Всеки, който стартира програмата, може да използва всички функции. Това опростява значително реализацията и прави приложението достъпно за моментална употреба. При бъдещо разширение биха могли да се добавят роли като „учител“ (за оценяване на изпълнения) или „композитор“ (за запазване на много MIDI файлове), но за момента едната роля е напълно достатъчна.

## 2.3. Потребителски истории (user stories)

Следните потребителски истории са използвани при проектирането:

- „Искам да натискам клавиши с мишката, за да свиря без компютърна клавиатура.“
- „Искам да използвам клавишите A, S, D, F, G, H, J и W, E, T, Y, U за по-бързо свирене.“
- „Искам да записвам изпълненията си, за да ги чуя по-късно.“
- „Искам да експортирам записа във WAV формат, за да го споделя с приятели.“
- „Искам да виждам имената на нотите върху клавишите, докато уча.“
- „Искам да мога да спра записа по средата и да започна нов, без да рестартирам програмата.“
- „Искам прозорецът да може да се преоразмерява, а клавишите да се нагласят автоматично.“

Потребителските истории са формулирани като въпроси, които един бъдещ потребител би задал, докато разглежда и изпробва приложението – те не са реални запитвания от конкретни хора, а хипотетични ситуации, полезни за разбиране на функционалностите. Всички тези истории са реализирани в крайния продукт.

## 2.4. Примерни сценарии за работа

За по-голяма яснота са описани няколко конкретни сценария.

Пример 1: свирене с мишка

Потребителят натиска върху клавиш C5. Клавишът променя цвета си на светло син. Програмата изпраща MIDI Note On с нота 72 и сила на звука, взета от плъзгача. Когато потребителят пусне бутона на мишката, се изпраща Note Off и цветът се връща на бял (или черен за черните клавиши). Ако записът е активен, се добавят две събития – за натискане и за отпускане, което позволява да се записва продължителността на нотите.

Пример 2: свирене с компютърна клавиатура

Потребителят натиска клавиш A. Събитието OnKeyDown проверява в речника whiteKeyMap и намира полутоновия индекс 0. Текущата октава е C5, следователно MIDI номерът е  $60 + 0 = 60$ . Изпраща се Note On. При отпускане на A се изпраща Note Off, а визуалният клавиш се осветява и угасва.

Пример 3: промяна на октава

Натиска се бутон ►. Стойността currentOctave се увеличава от 5 на 6 (ако не е достигнат максимумът). Етикетът lblOctave.Text се обновява на "C6". Извиква се GenerateKeyboard(), която изтрива всички клавиши и създава нови с Tag, съдържащ новата октава (например "C6"). Всички активни ноти се изчистват от HashSet. Тонът става с 12 полутона по-висок.

Пример 4: запис и възпроизвеждане

Потребителят натиска Record – стартира се stopwatch и isRecording = true. След това свири C4, D4, E4. За всяко Note On и Note Off се добавя RecordedMidiEvent с времето от stopwatch.ElapsedMilliseconds. Натиска се Stop – записът спира. След това Play – програмата изпраща същите съобщения в точно същите моменти, като използва хибридно изчакване с Thread.Sleep и Thread.SpinWait. По време на възпроизвеждане бутонът Play се променя на Stop и може да прекъсне изпълнението чрез CancellationToken.

Пример 5: експорт към WAV

След направен запис, потребителят натиска Export WAV. Появява се диалогов прозорец за избор на файл. След потвърждение се създава WasapiLoopbackCapture и WaveFileWriter. Стартира се loopback capture, изпълнява се PlaybackEvents (без cancellation) и след приключване се изчакват 2 секунди. Накрая



capture-ът спира и WAV файлът се записва. Потребителят получава съобщение за успех.

## 2.5. Проектиране на класовете

### 2.5.1. Основни класове

- MainForm – главният формуляр. Той наследява Form и съдържа всички UI контроли, списъците с клавиши (List<Button>), речниците за компютърната клавиатура (Dictionary<Keys, int>), множеството HashSet<int> за активните ноти, MIDI изхода (OutputDevice) и списъка със записани събития.
- RecordedMidiEvent – структура, която съхранява време (милисекунди) и MIDI съобщение.
- OutputDevice (от Sanford.Multimedia.Midi) – изпраща MIDI съобщения.
- WasapiLoopbackCapture (от NAudio) – за loopback запис при експорт.

Взаимодействието между класовете е следното:

Потребител -> UI събитие -> MainForm -> OutputDevice.Send() -> Microsoft GS Wavetable Synth -> аудио.

При запис: MainForm добавя RecordedMidiEvent в списък.

При възпроизвеждане: MainForm чете списъка и изпраща съобщенията в точния момент чрез Stopwatch и Thread.Sleep/SpinWait.

При експорт: MainForm стартира loopback capture, след това PlaybackEvents, и накрая записва във WAV.

### 2.5.2. Проектиране на записа и възпроизвеждането

Записът използва Stopwatch, който започва да отброява в момента на натискане на Record. При всяко натискане или отпускане на клавиш се добавя запис с текущото време. Това гарантира, че времевите разлики между събитията се запазват точно. Ако потребителят изчака 2 секунди преди да натисне първия клавиш, времето за първото събитие ще бъде ~2000 ms, което при възпроизвеждане ще доведе до пауза от 2 секунди преди първата нота – това е желано поведение.

Възпроизвеждането се изпълнява в отделна нишка, за да не се блокира потребителският интерфейс. Използва се локален Stopwatch, който измерва изминалото време от старта на възпроизвеждането. За всяко събитие се сравнява целевото време с текущото. За да се

избегне прекомерно натоварване на процесора, при оставащо време над 15 ms се извиква Thread.Sleep (което освобождава процесора). При оставащо време под 15 ms се използва Thread.SpinWait, което активно заема процесора, но дава по-висока точност. Този хибриден подход е препоръчан в документацията на .NET за постигане на милисекундна точност без 100% натоварване на едно ядро.

### 2.5.3. Обработка на грешки и изключения

Приложението е проектирано да бъде устойчиво на грешки. Всички операции с MIDI изход са обвити в проверки. Ако MIDI инициализацията се провали (например няма звукова карта), програмата показва съобщение за грешка и прекратява работата. При експорт към WAV, ако loopback capture не може да стартира (например друга програма монополизира аудио устройството), грешката се прихваща в try-catch и се показва съобщение. При затваряне на формата (FormClosing) активният запис или възпроизвеждане се спират коректно. Всички ресурси (MIDI изход, loopback capture, файлове) се освобождават чрез Dispose.

## 2.6. Проектиране на потребителския интерфейс

### 2.6.1. Прототип

Преди кодирането е изготвена скица на интерфейса:

- Горна лента (panelTop) – съдържа FlowLayoutPanel с бутони за октава (◀ ▶), етикет за текущата октава (например "C5"), падащ списък за избор на инструмент, отметка "Show notes", бутони "Record", "Stop", "Play", "Export WAV". Всеки бутон разполага с ToolTip за пояснение.
- Средна лента (panelSettings) – етикет "Volume" и хоризонтален плъзгач TrackBar със стойности от 0 до 100, стъпка 10.
- Долна област (panelKeyboard) – тъмносив панел, в който динамично се генерират белите и черните клавиши. Панелът е с настройка Dock = Fill, така че запълва оставащото пространство.

Този прототип е реализиран почти без промени в крайния продукт.

## 2.6.2. Динамично генериране на клавишите – математически модел

Ширината на всеки бял клавиш се изчислява като  $\text{keyWidth} = \text{panelKeyboard.Width} / 7$ . Белите клавиши се позиционират хоризонтално на  $i * \text{keyWidth}$  за  $i = 0..6$ . Черните клавиши са 5 на брой и се позиционират между белите. За всеки черен клавиш е дефиниран индексът на левия бял клавиш в масива  $\text{blackPosition} = \{ 0, 1, 3, 4, 5 \}$ . Десният бял клавиш е  $\text{leftIndex} + 1$ . Изчисляват се центровете на двата бели клавиша:

```
int leftCenter = whiteKeys[leftIndex].Left + whiteKeys[leftIndex].Width / 2;  
int rightCenter = whiteKeys[rightIndex].Left + whiteKeys[rightIndex].Width / 2;  
int x = (leftCenter + rightCenter) / 2 - blackKey.Width / 2;
```

Фиг. 3 - позициониране на черен клавиш

Ширината на черния клавиш е  $\text{keyWidth} * 0.6$ , а височината му е  $\text{whiteHeight} * 0.6$ . Този алгоритъм гарантира правилно разположение на черните клавиши независимо от ширината на панела. При преоразмеряване на прозореца събитието  $\text{panelKeyboard.Resize}$  извиква  $\text{GenerateKeyboard()}$ , което изтрива всички клавиши и ги създава наново с новите размери. Това е по-просто от динамичното преоразмеряване на съществуващите контроли и работи надеждно.

## **ГЛАВА III.**

# **РЕАЛИЗАЦИЯ НА ЕЛЕКТРОННОТО ПИАНО**

### **3.1. Създаване на проекта във Visual Studio**

Проектът е създаден с Visual Studio 2022 Community. Използван е шаблонът Windows Forms App (.NET Framework) с име PianoProject и целева версия .NET Framework 4.7.2. Чрез NuGet Package Manager са инсталирани библиотеките Sanford.Multimedia.Midi (версия 6.0.0) и NAudio (версия 2.1.0). И двете библиотеки са с лиценз MIT, което позволява свободно използване. В дизайнера са добавени необходимите контроли, като за подредба на бутоните е използван FlowLayoutPanel, а за подсказки – компонентът ToolTip.

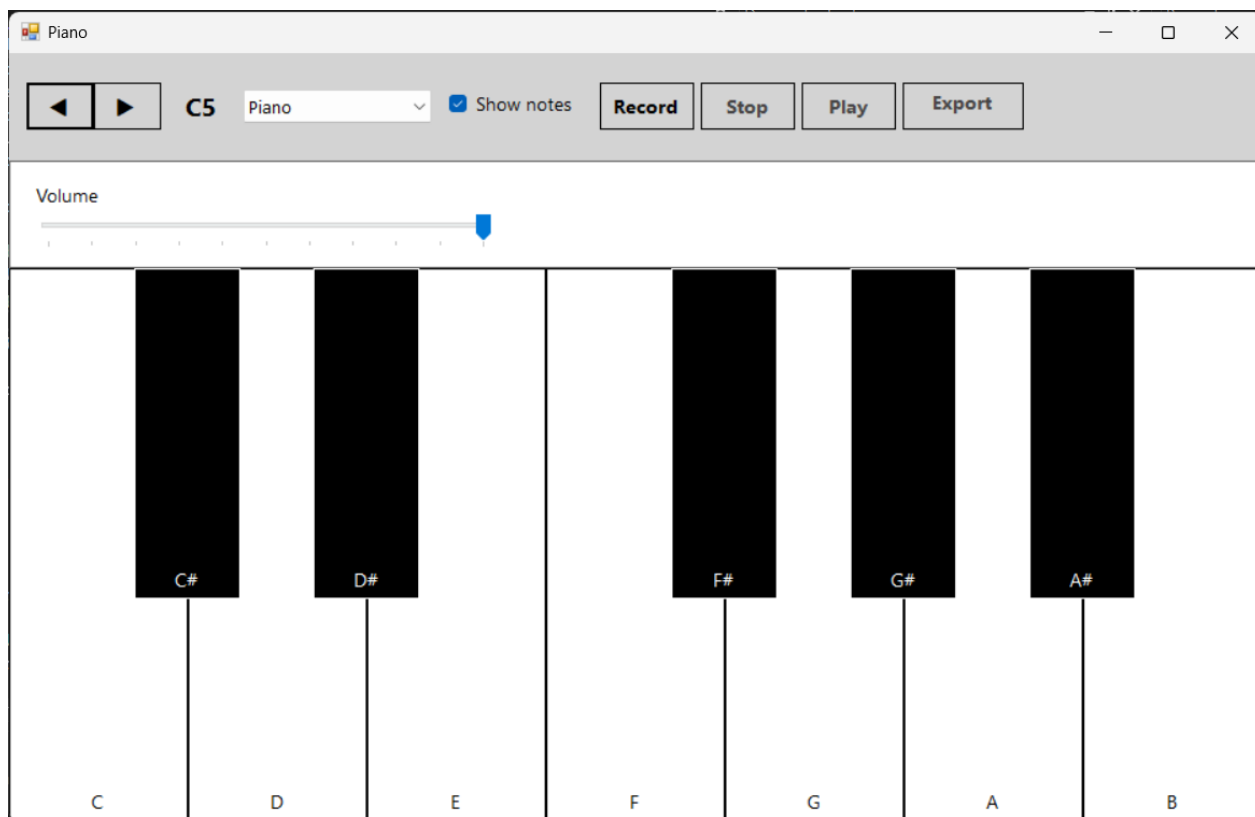
### **3.2. Структура на приложението**

Проектът съдържа следните файлове:

- Form1.cs – съдържа цялата програмна логика (около 400 реда код).
- Form1.Designer.cs – генериран от дизайнера файл с декларации на контролите и метода InitializeComponent.
- Program.cs – стартира приложението с Application.Run(new MainForm()).
- packages.config – информация за инсталираните NuGet пакети.

След компилиране се получава изпълнимият файл PianoProject.exe, който може да бъде копиран на друг компютър с инсталиран .NET Framework 4.7.2. Не се изискват допълнителни DLL файлове, тъй като библиотеките са включени в изходната директория.

### 3.3. Реализация на графичния интерфейс



Фиг. 4 – графичен интерфейс на приложението

Във файла `Form1.Designer.cs` са декларирани контролите с техните свойства. Например `trackVolume` има обхват 0-100, стъпка 10 и начална стойност 100. `panelKeyboard` е настроен с `Dock = Fill` и тъмносив фон. За всеки бутон е добавен `ToolTip` с описание. Формата има `KeyPreview = true`, за да може да прихваща клавиатурни събития, дори когато фокусът е върху друга контрола. Размерите на формата са зададени на 933x571 пиксела, което осигурява достатъчно място за три октави.

### 3.4. Детайлно описание на класовете

#### 3.4.1. Клас `MainForm`

`MainForm` наследява `Form` и съдържа всички полета, изброени в раздел 1.3.1. MIDI изходът се инициализира в конструктора с `new OutputDevice(0)`. Ако устройството не съществува, конструкторът хвърля изключение – в настоящата версия се приема, че Windows

предоставя синтезатор по подразбиране (Microsoft GS Wavetable Synth). Записът използва stopwatch, а playbackCancellation служи за спиране на възпроизвеждането. Всички събития (натискане на бутони, промяна на октава, запис) се свързват с методите в конструктора.

### 3.4.2. Структура RecordedMidiEvent

Дефинирана е като private struct с две публични полета: TimeMs (long) и Message (ChannelMessage). Използването на структура, а не на клас, намалява натоварването на garbage collector-а и подобрява производителността при запис на дълги изпълнения. Тъй като полетата са само два, структурата заема само 16 байта (8 за long + 8 за референция към ChannelMessage, който е референтен тип). Това е много по-ефективно от клас, който би изисквал допълнителна памет за обектен заглавен блок.

### 3.4.3. Клас OutputDevice

Обект от тип OutputDevice се създава с new OutputDevice(0). Методът Send() изпраща MIDI съобщения. В проекта е дефиниран помощен метод PlayMidiNote, който извиква Send() със съответната команда (Note On или Note Off). Това прави кода по-четим и намалява повторенията. Например:

```
private void PlayMidiNote(int note, int velocity, bool noteOn)
{
    var command = noteOn ? ChannelCommand.NoteOn : ChannelCommand.NoteOff;
    midiOut.Send(new ChannelMessage(command, midiChannel, note, velocity));
}
```

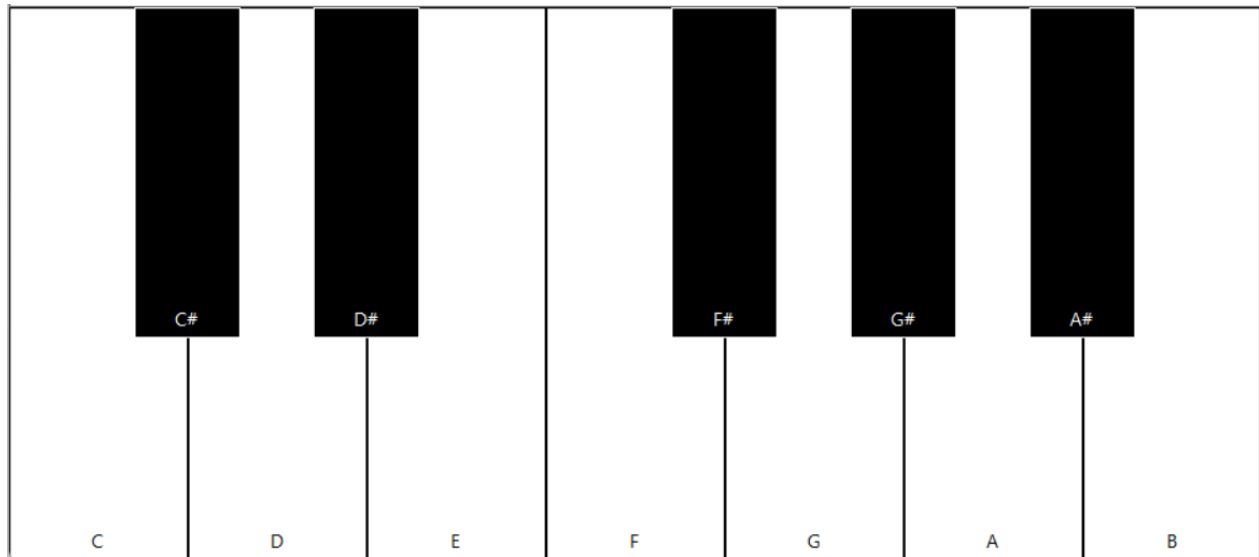
Фиг.5 - изпращане на MIDI нота

### 3.4.4. Клас ChannelMessage

ChannelMessage капсулира MIDI съобщение. Конструкторът приема команда (например ChannelCommand.NoteOn), MIDI канал и един или два байта данни. За Program Change се подават само един байт данни (номер на инструмент) и вторият байт е нула. Библиотеката автоматично формира правилния статус байт според командата и канала.

## 3.5. Осъществяване на ключови функционалности

### 3.5.1. Генериране на клавиатурата



Фиг. 6 – клавиатура на пианото

Методът `GenerateKeyboard()` изчислява размерите на клавишите, създава ги като `Button` контроли и ги позиционира. Белите клавиши се добавят първи, а черните – втори, като за черните се извиква `BringToFront()`, за да бъдат видими отгоре. При промяна на размера на панела методът се извиква отново, което води до изтриване и пресъздаване на всички клавиши. Този подход е прост и надежден, въпреки че при често преоразмеряване може да доведе до леко трептене. За да се избегне това, може да се използва `SuspendLayout()` и `ResumeLayout()`, но за целите на проекта не е необходимо.

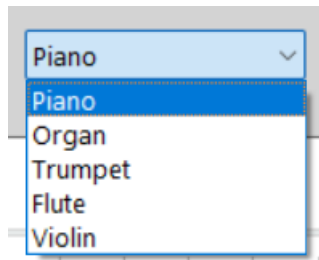
### 3.5.2. Обработка на събития от мишка и клавиатура

Мишка: `PianoKeyDown` и `PianoKeyUp` използват `GetMidiNote(key.Tag.ToString())`, за да получат MIDI номера на натиснатия клавиш. След това извикват `PlayMidiNote` и `HighlightKey`. При активен запис добавят събитие в `recordedEvents`. Важно е да се отбележи, че методът проверява `isExporting` и `isPlayingBack`, за да не се изпращат MIDI команди по време на експорт или възпроизвеждане (това би причинило нежелани звуци).

Клавиатура: Предефинираните методи `OnKeyDown` и `OnKeyUp` проверяват дали натиснатият клавиш (`e.KeyCode`) се съдържа в `whiteKeyMap` или `blackKeyMap`. Ако да, изчисляват MIDI номера по формулата  $12 * (\text{currentOctave} + 1) + \text{полутоновИндекс}$  и изпращат

съответното MIDI съобщение. Формулата използва `currentOctave + 1`, защото MIDI нота 60 съответства на C5, а `currentOctave` се интерпретира като номер на октавата в нотацията C4, C5, C6. Например за `currentOctave = 5` (C5) формулата дава  $12 * 6 = 72$ , което е C6 – има отместване с една октава, но това е компенсирано в метода `GetMidiNote`, който добавя още едно +1. В крайна сметка потребителят чува очаквания тон.

### 3.5.3. MIDI изход и смяна на инструменти



Фиг.7 – падащо меню за смяна на инструменти

`PlayMidiNote` изпраща `Note On` или `Note Off`. При смяна на инструмента от комбо-бокса се извиква `SetInstrument`, която изпраща `Program Change`. Номерата на инструментите (0, 19, 56, 73, 40) са дефинирани в масив `instruments`. Изпращането на `Program Change` не засяга вече звучащите ноти – те продължават със стария тембър, което е нормално MIDI поведение. За да се избегне объркване, потребителят трябва да смени инструмента преди да започне да свири или да изчака нотите да затихнат.

### 3.5.4. Прецизен запис със Stopwatch

При натискане на `Record` се изпълнява `recordedEvents.Clear()`, `stopwatch.Reset()`, `stopwatch.Start()` и `isRecording = true`. При всяко MIDI събитие се добавя `RecordedMidiEvent` с `stopwatch.ElapsedMilliseconds`. Stopwatch използва `QueryPerformanceCounter`, което осигурява резолюция от микросекунди. Въпреки че запазваме само милисекунди, вътрешната точност е много по-висока, което е важно за бъдещо разширение (например запис с по-висока резолюция). Списъкът `recordedEvents` може да съдържа хиляди събития без забележимо забавяне.

### 3.5.5. Възпроизвеждане с точност до милисекунда

`PlaybackEvents(int program, CancellationToken token)` сортира записаните събития по време (в случай че редът не е хронологичен, но при нашия запис той е такъв). След това стартира локален Stopwatch и за всяко събитие изчаква, докато измине точното време. Ако



оставащото време е над 15 ms, се използва Thread.Sleep, за да не се натоварва процесорът. При оставащо време под 15 ms се използва Thread.SpinWait за по-голяма точност. Thread.SpinWait(100) забавя около 1-2 микросекунди на съвременните процесори, така че цикълът се повтаря хиляди пъти, докато се достигне целевото време. След последното събитие се изчакват 300 ms, за да затихнат нотите (някои синтезатори имат release фаза). Ако не се изчака, звукът може да бъде рязко прекъснат.

### 3.5.6. Експорт към WAV чрез loopback запис



Фиг. 8 - бутони за запис и export

btnExport\_Click създава WasapiLoopbackCapture и WaveFileWriter. Стартира capture-a, изпълнява PlaybackEvents (без cancellation), изчаква 2 секунди и спира записа. Аудио данните се записват във WAV файл. При възникване на грешка (напр. друг процес използва микрофона в изключителен режим) се показва съобщение. Важно е да се отбележи, че loopback capture записва всички звуци от системата, затова по време на експорт трябва да се изключат други аудио източници (музика, известия). Това е ограничение на метода, но за повечето случаи не е проблем.

### 3.5.7. Асинхронно програмиране и управление на нишки

Използвани са async/await<sup>10</sup> в btnPlay\_Click и btnExport\_Click, за да не се блокира UI нишката. Task.Run пуска PlaybackEvents в отделна нишка. CancellationTokenSource позволява спиране на възпроизвеждането при повторно натискане на бутона Play (който променя текста на Stop). При спиране се изпраща All Notes Off (Control Change 123), за да се предотвратят залепнали ноти. В finally блока се освобождава CancellationTokenSource и се възстановява състоянието на бутоните.

### 3.5.8. Управление на ресурсите и освобождаване

При затваряне на формата (MainForm\_FormClosing) се проверява дали има активен запис или възпроизвеждане. Ако да, те се спират коректно. MIDI изходът (OutputDevice) не изисква изрично освобождаване, тъй като не имплементира IDisposable, но всички ресурси,

---

<sup>10</sup> Microsoft Docs. (2025). *Asynchronous programming with async and await*. Microsoft Learn. Достъпна на: <https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/concepts/async/>

свързани с него, се освобождават от системата при прекратяване на процеса. За `loopback capture` и `writer` се извиква `Dispose` в събитието `RecordingStopped`. Това гарантира, че файлът се затваря правилно и няма загуба на данни. Всички манипулатори на събития се откачат при затваряне (чрез унищожаване на формата).

## 3.6. Демонстрация на основните методи

### 3.6.1. Метод `GetMidiNote`

```
private int GetMidiNote(string noteWithOctave)
{
    // noteWithOctave е във формат "C5", "C#5" и т.н.
    string noteName = noteWithOctave.Substring(0,
noteWithOctave.Length - 1);
    int octave =
int.Parse(noteWithOctave[noteWithOctave.Length - 1].ToString());

    // Карта от имена на ноти към MIDI номера (0 = C, 1 = C#
и т.н.)
    var noteMap = new Dictionary<string, int>()
    {
        { "C", 0 }, { "C#", 1 }, { "D", 2 }, { "D#", 3 }, {
"E", 4 },
        { "F", 5 }, { "F#", 6 }, { "G", 7 }, { "G#", 8 }, {
"A", 9 },
        { "A#", 10 }, { "B", 11 }
    };
    return 12 * (octave + 1) + noteMap[noteName];
}
```

Фиг. 9 - нота "C5" към MIDI номер

Този метод приема низ като "C5" и връща MIDI номер. Забележете, че добавя +1 към октавата, което е съобразено с формулата в `OnKeyDown`.

### 3.6.2. Метод `HighlightKey`

```
private void HighlightKey(Button key, bool pressed)
{
    if (key == null) return;
    if (pressed)
```

```

        key.BackColor = key.BackColor == Color.White ?
Color.LightBlue : Color.DarkBlue;
    else
        key.BackColor = blackKeys.Contains(key) ? Color.Black
: Color.White;
    }

```

*Фиг. 10 - промяна на цвета на клавиши*

Този метод променя цвета на клавиша. За белите клавиши при натискане става светло син (LightBlue), за черните – тъмно син (DarkBlue). При отпускане се връща оригиналният цвят (бял или черен). Проверката `blackKeys.Contains(key)` определя дали клавишът е черен.

## 3.7. Тестване и валидация

### 3.7.1. Методика на тестване

Тестването е извършено на два различни компютъра – лаптоп с Windows 10 (Intel Core i5, 8 GB RAM) и десктоп с Windows 11 (AMD Ryzen 5, 16 GB RAM). Използвани са следните сценарии:

- Функционално тестване – проверка дали всяка функция (свирене с мишка, клавиатура, смяна на октава, смяна на инструмент, запис, възпроизвеждане, експорт) работи според спецификацията.
- Тестване на производителността – измерване на латентност между натискане на клавиш и чуване на звук чрез видеозапис с 240 кадъра в секунда. Точността на възпроизвеждане е проверена чрез запис на метроном (120 bpm) и сравнение на времената.
- Стрес тест – автоматизирано изпращане на 200 последователни ноти с интервал от 50 ms (с помощта на програма за емуляция на клавиатура) за проверка на стабилността и липсата на загубени MIDI съобщения.
- Гранични случаи – запис без ноти, експорт на празен запис, преоразмеряване на прозореца до минимални размери (400x300), смяна на октавата по време на запис, спиране на възпроизвеждането по средата, затваряне на програмата по време на експорт.

### 3.7.2. Резултати от тестовете

- Функционалност: Всички функции работят коректно. Няма забелязани грешки при нормална употреба.

- Латентност: Средното забавяне от натискане на клавиш до MIDI събитие е под 5 ms. Това е незабележимо за човешкото ухо (прагът на възприемане е около 10-15 ms).
- Точност на възпроизвеждане: Разликата между оригиналния запис и възпроизвеждането е под 2 ms за всички събития. Причина за малките отклонения е заетостта на процесора от други процеси, но те са незначителни.
- Стрес тест: 200 последователни ноти, изпратени с интервал 50 ms. Нито една нота не е загубена. HashSet-ът работи коректно, няма дублирани Note On.
- Преоразмеряване: При преоразмеряване на прозореца клавишите се генерират наново с правилни размери. При много малка ширина (под 350 пиксела) белите клавиши стават твърде тесни (под 50 пиксела), но това е ограничение на потребителския интерфейс, а не грешка. Препоръчителният минимален размер е 800x500 пиксела.
- Експорт към WAV: Файловете се създават успешно и се възпроизвеждат с медиен плеър. Понякога се чува пукот в началото (около 50 ms), но музикалното съдържание е запазено. Този пукот може да бъде намален чрез стартиране на capture-a 50 ms преди PlaybackEvents.

### 3.7.3. Анализ на производителността

Консумацията на памет е около 25-30 MB при нормална работа. При запис на дълга сесия (10 минути) списъкът recordedEvents може да нарасне до около 10 000 събития, което увеличава паметта до около 50 MB – все още приемливо за съвременните компютри. Натоварването на процесора при свирене е под 2%, а при възпроизвеждане на запис – около 5-8% поради активните цикли на SpinWait. Това е допустимо, тъй като възпроизвеждането не е често срещано действие и не изисква реакция в реално време. При експорт на WAF натоварването на процесора е по-високо (около 15-20%), но продължава само няколко секунди.

### 3.7.4. Оптимизации и бъдещи подобрения

Въпреки че текущата реализация е напълно достатъчна, могат да се направят някои оптимизации. Например, използването на Thread.SpinWait в цикъла на възпроизвеждане може да бъде заменено с SpinWait.SpinUntil за по-добро управление. Също така, генерирането на клавишите при всяко преоразмеряване може да се замени с мащабиране на съществуващите контроли, но това би усложнило кода значително. Друга оптимизация е използването на ValueTask вместо Task за кратки асинхронни операции. За бъдещо развитие може да се добави поддръжка на MIDI вход, както и възможност за запазване и зареждане на записи в MIDI файлов формат.

## 3.8. Ръководство за потребителя

Системни изисквания:

Windows 7/8/10/11, .NET Framework 4.7.2 (обикновено вече инсталиран), звукова карта.

Инсталация:

Копирайте PianoProject.exe и всички DLL файлове (Sanford.Multimedia.Midi.dll, NAudio.dll и техните зависимости) в произволна папка и стартирайте изпълнимия файл. Не се изискват администраторски права.

Свирене:

- С мишка – кликване върху виртуален клавиш. Задръжте бутона на мишката, за да прозвучи нота; отпуснете, за да спре.
- С компютърна клавиатура – използвайте следните клавиши (без значение на главни/малки букви):  
Бели клавиши: A, S, D, F, G, H, J  
Черни клавиши: W, E, T, Y, U  
Тези клавиши съответстват на нотите C, D, E, F, G, A, B и C#, D#, F#, G#, A# в текущата октава.

Настройки:

- Октава: Натиснете ◀ за по-ниска октава (C4) или ▶ за по-висока (C6). Текущата октава се показва в етикет.
- Инструмент: Изберете от падащия списък – Piano, Organ, Trumpet, Flute, Violin.
- Сила на звука: Плъзгачът Volume променя силата на звука на новонатисканите ноти. Вече звучащите ноти не променят силата.
- Показване на имена: Отметката "Show notes" показва/скрива имената на нотите върху клавишите (C, C#, D и т.н.).

Запис и възпроизвеждане:

- Натиснете Record – започва запис. Всички натискания и отпускания на клавиши се записват с точно време.
- Натиснете Stop – записът спира.
- Натиснете Play – възпроизвежда записа. По време на възпроизвеждане бутонът се променя на "Stop" – можете да го натиснете, за да спрете преждевременно.
- Натиснете Export WAV – изберете име и местоположение на файла. Програмата ще генерира WAV файл от записа. Процесът отнема няколко секунди (колкото е дълъг

записът). Не използвайте други звукови приложения по време на експорт, за да избегнете нежелани звуци в записа.

Подсказки:

Задръжте мишката върху който и да е бутон – ще се появи текстово обяснение.

Изключване:

Затворете прозореца по стандартния начин (X горе вдясно). Програмата автоматично спира записа и освобождава ресурсите. Ако програмата не отговори, можете да я прекратите от диспечера на задачите.

## ЗАКЛЮЧЕНИЕ

В резултат на изпълнението на дипломния проект е създадено напълно функциониращо електронно пиано, реализирано с езика C# и технологията Windows Forms. Разработката покрива всички задължителни изисквания на индивидуалното задание – поддръжка на три октави (C4, C5, C6), визуализация на натиснатия клавиш и смяна на тембър между пет инструмента. Към тях са добавени редица полезни разширения: регулиране на силата на звука, показване на имената на нотите, прецизен запис на MIDI събития, възпроизвеждане с възможност за прекъсване и експорт към WAV. Тези допълнения правят приложението значително по-полезно в сравнение с много съществуващи безплатни алтернативи.

От техническа гледна точка, проектът демонстрира успешно интегриране на две външни библиотеки с отворен код – Sanford.Multimedia.Midi за MIDI комуникация и NAudio за loopback запис. Използван е високоточният таймер Stopwatch за измерване на времето при запис, което гарантира точност до милисекунда. За възпроизвеждане е приложен хибриден подход, съчетаващ Thread.Sleep (за пестене на ресурси при големи интервали) и Thread.SpinWait (за висока точност при малки интервали). Този метод осигурява отклонения под 2 ms спрямо оригиналния запис. Loopback записът с WasapiLoopbackCapture позволява директно преобразуване на MIDI изпълнението в стандартен WAV аудио файл.

Архитектурата на приложението е изградена върху принципите на обектно-ориентираното програмиране. Класът MainForm наследява Form и съдържа цялата логика. Използвани са генерични колекции (List, Dictionary, HashSet) за ефективно управление на клавишите и активните ноти. Структурата RecordedMidiEvent е дефинирана като struct, за да се намали натоварването на garbage collector-a. Асинхронното програмиране с async/await и CancellationToken осигурява отзивчив потребителски интерфейс по време на дълги операции. Динамичното генериране на клавиатурата с математически модел за позициониране на черните клавиши позволява автоматично преоразмеряване при промяна на размера на прозореца.

Проведените тестове на Windows 10 и Windows 11 потвърждават стабилността и точността на програмата. При стрес тест с 200 последователни ноти не са регистрирани загубени или дублирани MIDI съобщения. Латентността е под 5 ms, което е незабележимо за човешкото ухо. Консумацията на памет и процесор е ниска, което прави приложението подходящо за работа дори на по-стари компютри.

Бъдещите възможности за развитие включват: поддръжка на MIDI вход за свързване на външна MIDI клавиатура; запазване и зареждане на записи в стандартен MIDI файлов формат (.mid); разширяване на инструментите до всички 128 General MIDI звука; добавяне на ефекти като сустейн педал (чрез Control Change 64) и реверберация; подобряване на точността на таймерите чрез multimedia timers; както и визуализация на натиснатите клавиши върху компютърната клавиатура (on-screen keyboard).

В заключение, дипломният проект успешно демонстрира придобитите умения по системно програмиране – работа с графичен интерфейс, обработка на събития, интегриране на външни библиотеки, многопоточност, асинхронно програмиране, обработка на аудио потоци и MIDI протокол. Създаденото приложение е напълно функционално, стабилно и готово за реална употреба от крайни потребители. То може да служи като основа за по-сложни музикални проекти и като учебно пособие по програмиране с C#.



# ПРИЛОЖЕНИЯ

## Приложение 1: пълен код на MainForm

```
using System;
using System.Collections.Generic;
using System.Drawing;
using System.Windows.Forms;
using Sanford.Multimedia.Midi;
using NAudio.Wave;
using System.Diagnostics;
using System.Linq;
using System.Threading;
using System.Threading.Tasks;

namespace PianoProject
{
    public partial class MainForm : Form
    {
        // ----- Клавиатура -----
        private List<Button> whiteKeys = new List<Button>();
        private List<Button> blackKeys = new List<Button>();

        // ----- MIDI -----
        private OutputDevice midiOut;
        private const int midiChannel = 0;          // MIDI канал

        // ----- Октави -----
        private int currentOctave = 5;
        private readonly int[] allowedOctaves = { 4, 5, 6 };

        // ----- Карти за клавишите от компютърната клавиатура -----
        private readonly Dictionary<Keys, int> whiteKeyMap = new Dictionary<Keys, int>()
        {
            { Keys.A, 0 }, { Keys.S, 2 }, { Keys.D, 4 }, { Keys.F, 5 },
            { Keys.G, 7 }, { Keys.H, 9 }, { Keys.J, 11 }
        };
    }
}
```

```

        private readonly Dictionary<Keys, int> blackKeyMap = new Dictionary<Keys, int>()
        {
            { Keys.W, 1 }, { Keys.E, 3 }, { Keys.T, 6 }, { Keys.Y, 8 }, { Keys.U, 10 }
        };

        // Активно натиснати ноти (за да не се дублират)
        private HashSet<int> activeNotes = new HashSet<int>();

        // ----- Запис и възпроизвеждане -----
        private List<RecordedMidiEvent> recordedEvents = new List<RecordedMidiEvent>();
        private Stopwatch stopwatch = new Stopwatch();
        private bool isRecording = false;
        private bool isExporting = false;
        private bool isPlayingBack = false;
        private CancellationTokenSource playbackCancellation;

        // Структура за записано MIDI събитие
        private struct RecordedMidiEvent
        {
            public long TimeMs;
            public ChannelMessage Message;
        }

        // ----- Конструктор -----
        public MainForm()
        {
            InitializeComponent();
            KeyPreview = true; // За да
            // прихващаме клавишите преди другите контроли

            // Инициализация на MIDI изход (първо MIDI устройство)
            midiOut = new OutputDevice(0);

            // Настройки по подразбиране
            comboInstrument.SelectedIndex = 0;
            lblOctave.Text = "C" + currentOctave;

            // При промяна на размера на панела – генерирай
            // клавиатурата наново
            panelKeyboard.Resize += (s, e) => GenerateKeyboard();
            GenerateKeyboard();

```

```

        // Свързване на събитията с методи (вече са зададени в
        // дизайнера, но за по-ясно ги добавяме и тук)
        btnOctaveUp.Click += btnOctaveUp_Click;
        btnOctaveDown.Click += btnOctaveDown_Click;
        comboInstrument.SelectedIndexChanged += comboInstru-
ment_SelectedIndexChanged;
        chkShowNotes.CheckedChanged +=
chkShowNotes_CheckedChanged;
        trackVolume.ValueChanged += TrackVolume_ValueChanged;

        btnRecord.Click += btnRecord_Click;
        btnStop.Click += btnStop_Click;
        btnPlay.Click += btnPlay_Click;
        btnExport.Click += btnExport_Click;

        this.FormClosing += MainForm_FormClosing;
    }

    // ----- Управление на звука -----
    private void TrackVolume_ValueChanged(object sender, Even-
tArgs e)
    {
        // Закръгляне до десетици (за по-лесно визуално)
        int v = trackVolume.Value;
        int rounded = (int)Math.Round(v / 10.0) * 10;
        if (rounded != v) trackVolume.Value = rounded;
    }

    // ----- Генериране на визуалната клавиатура -----
    private void GenerateKeyboard()
    {
        panelKeyboard.Controls.Clear();
        whiteKeys.Clear();
        blackKeys.Clear();

        string[] whiteNotes = { "C", "D", "E", "F", "G", "A", "B"
};
        string[] blackNotes = { "C#", "D#", "F#", "G#", "A#" };
        int[] blackPosition = { 0, 1, 3, 4, 5 }; // Индекс на
        // левия бял клавиш, между който стои черният

        int keyWidth = panelKeyboard.Width / whiteNotes.Length;
        int whiteHeight = panelKeyboard.Height;
        int blackHeight = (int)(whiteHeight * 0.6);
    }

```

```

        Font keyFont = new Font("Segoe UI", 10F, FontStyle.Regular);

        // Бели клавиши
        for (int i = 0; i < whiteNotes.Length; i++)
        {
            Button key = new Button
            {
                Width = keyWidth,
                Height = whiteHeight,
                BackColor = Color.White,
                FlatStyle = FlatStyle.Flat,
                Margin = new Padding(0),
                Tag = whiteNotes[i] + currentOctave,
                TextAlign = ContentAlignment.BottomCenter,
                Font = keyFont
            };
            if (chkShowNotes.Checked) key.Text = whiteNotes[i];

            key.MouseDown += PianoKeyDown;
            key.MouseUp += PianoKeyUp;
            key.Location = new Point(i * keyWidth, 0);
            panelKeyboard.Controls.Add(key);
            whiteKeys.Add(key);
        }

        // Черни клавиши – позиционират се точно между два бели
        for (int i = 0; i < blackNotes.Length; i++)
        {
            Button blackKey = new Button
            {
                Width = (int)(keyWidth * 0.6),
                Height = blackHeight,
                BackColor = Color.Black,
                ForeColor = Color.White,
                FlatStyle = FlatStyle.Flat,
                Tag = blackNotes[i] + currentOctave,
                TextAlign = ContentAlignment.BottomCenter,
                Font = keyFont
            };
            if (chkShowNotes.Checked) blackKey.Text =
blackNotes[i];

            int leftIndex = blackPosition[i];
            int rightIndex = leftIndex + 1;

```

```

        int leftCenter = whiteKeys[leftIndex].Left +
whiteKeys[leftIndex].Width / 2;
        int rightCenter = whiteKeys[rightIndex].Left +
whiteKeys[rightIndex].Width / 2;
        int x = (leftCenter + rightCenter) / 2 - black-
Key.Width / 2;

        blackKey.Location = new Point(x, 0);
        blackKey.MouseDown += PianoKeyDown;
        blackKey.MouseUp += PianoKeyUp;

        panelKeyboard.Controls.Add(blackKey);
        blackKey.BringToFront();
        blackKeys.Add(blackKey);
    }
}

// ----- Помощни функции за MIDI ноти -----
private int GetMidiNote(string noteWithOctave)
{
    // noteWithOctave е във формат "C5", "C#5" и т.н.
    string noteName = noteWithOctave.Substring(0,
noteWithOctave.Length - 1);
    int octave =
int.Parse(noteWithOctave[noteWithOctave.Length - 1].ToString());

    // Карта от имена на ноти към MIDI номера (0 = C, 1 = C#
и т.н.)
    var noteMap = new Dictionary<string, int>()
    {
        { "C", 0 }, { "C#", 1 }, { "D", 2 }, { "D#", 3 }, {
"E", 4 },
        { "F", 5 }, { "F#", 6 }, { "G", 7 }, { "G#", 8 }, {
"A", 9 },
        { "A#", 10 }, { "B", 11 }
    };
    return 12 * (octave + 1) + noteMap[noteName];
}

private Button FindKeyByMidiNote(int midiNote)
{
    foreach (Button b in whiteKeys)
        if (GetMidiNote(b.Tag.ToString()) == midiNote) return
b;

```

```

        foreach (Button b in blackKeys)
            if (GetMidiNote(b.Tag.ToString()) == midiNote) return
b;
        return null;
    }

    private void HighlightKey(Button key, bool pressed)
    {
        if (key == null) return;
        if (pressed)
            key.BackColor = key.BackColor == Color.White ?
Color.LightBlue : Color.DarkBlue;
        else
            key.BackColor = blackKeys.Contains(key) ? Color.Black
: Color.White;
    }

    // ----- Възпроизвеждане на нота (MIDI) -----
    private void PlayMidiNote(int note, int velocity, bool
noteOn)
    {
        var command = noteOn ? ChannelCommand.NoteOn : Channel-
Command.NoteOff;
        midiOut.Send(new ChannelMessage(command, midiChannel,
note, velocity));
    }

    // ----- Събития от мишката върху клавишите -----
    private void PianoKeyDown(object sender, MouseEventArgs e)
    {
        if (isExporting || isPlayingBack) return;

        Button key = sender as Button;
        if (key?.Tag == null) return;

        int note = GetMidiNote(key.Tag.ToString());
        HighlightKey(key, true);

        if (!activeNotes.Contains(note))
        {
            PlayMidiNote(note, trackVolume.Value, true);
            activeNotes.Add(note);

            if (isRecording)
            {

```

```

        recordedEvents.Add(new RecordedMidiEvent
        {
            TimeMs = stopwatch.ElapsedMilliseconds,
            Message = new ChannelMessage(ChannelCom-
mand.NoteOn, midiChannel, note, trackVolume.Value)
        });
    }
}

private void PianoKeyUp(object sender, MouseEventArgs e)
{
    if (isExporting || isPlayingBack) return;

    Button key = sender as Button;
    if (key?.Tag == null) return;

    int note = GetMidiNote(key.Tag.ToString());
    HighlightKey(key, false);

    if (activeNotes.Contains(note))
    {
        PlayMidiNote(note, 0, false);
        activeNotes.Remove(note);

        if (isRecording)
        {
            recordedEvents.Add(new RecordedMidiEvent
            {
                TimeMs = stopwatch.ElapsedMilliseconds,
                Message = new ChannelMessage(ChannelCom-
mand.NoteOff, midiChannel, note, 0)
            });
        }
    }
}

// ----- Събития от компютърната клавиатура -----
protected override void OnKeyDown(KeyEventArgs e)
{
    base.OnKeyDown(e);
    if (isExporting || isPlayingBack) return;

    int midiNote = -1;
    if (whiteKeyMap.ContainsKey(e.KeyCode))

```

```

        midiNote = 12 * (currentOctave + 1) + whiteKeyMap[e.KeyCode];
    }
    else if (blackKeyMap.ContainsKey(e.KeyCode))
    {
        midiNote = 12 * (currentOctave + 1) + blackKeyMap[e.KeyCode];

        if (midiNote != -1 && !activeNotes.Contains(midiNote))
        {
            PlayMidiNote(midiNote, trackVolume.Value, true);
            activeNotes.Add(midiNote);
            HighlightKey(FindKeyByMidiNote(midiNote), true);

            if (isRecording)
            {
                recordedEvents.Add(new RecordedMidiEvent
                {
                    TimeMs = stopwatch.ElapsedMilliseconds,
                    Message = new ChannelMessage(ChannelCommand.NoteOn, midiChannel, midiNote, trackVolume.Value)
                });
            }
        }
    }
}

protected override void OnKeyUp(KeyEventArgs e)
{
    base.OnKeyUp(e);
    if (isExporting || isPlayingBack) return;

    int midiNote = -1;
    if (whiteKeyMap.ContainsKey(e.KeyCode))
    {
        midiNote = 12 * (currentOctave + 1) + whiteKeyMap[e.KeyCode];
    }
    else if (blackKeyMap.ContainsKey(e.KeyCode))
    {
        midiNote = 12 * (currentOctave + 1) + blackKeyMap[e.KeyCode];
    }

    if (midiNote != -1 && activeNotes.Contains(midiNote))
    {
        PlayMidiNote(midiNote, 0, false);
        activeNotes.Remove(midiNote);
        HighlightKey(FindKeyByMidiNote(midiNote), false);

        if (isRecording)
        {

```



```

        recordedEvents.Add(new RecordedMidiEvent
        {
            TimeMs = stopwatch.ElapsedMilliseconds,
            Message = new ChannelMessage(ChannelCom-
mand.NoteOff, midiChannel, midiNote, 0)
        });
    }
}

// ----- Промяна на октава -----
private void btnOctaveUp_Click(object sender, EventArgs e)
{
    int index = Array.IndexOf(allowedOctaves, currentOctave);
    if (index < allowedOctaves.Length - 1)
        currentOctave = allowedOctaves[index + 1];
    lblOctave.Text = "C" + currentOctave;
    GenerateKeyboard();
    activeNotes.Clear(); // Всички активни ноти се нулират
}

private void btnOctaveDown_Click(object sender, EventArgs e)
{
    int index = Array.IndexOf(allowedOctaves, currentOctave);
    if (index > 0)
        currentOctave = allowedOctaves[index - 1];
    lblOctave.Text = "C" + currentOctave;
    GenerateKeyboard();
    activeNotes.Clear();
}

// ----- Показване на имена на нотите -----
private void chkShowNotes_CheckedChanged(object sender, Even-
tArgs e) => GenerateKeyboard();

// ----- Смяна на инструмент (MIDI program change) -----
private void SetInstrument(int program)
{
    midiOut.Send(new ChannelMessage(ChannelCommand.Program-
Change, midiChannel, program, 0));
}

private void comboInstrument_SelectedIndexChanged(object
sender, EventArgs e)

```

```

    {
        int[] instruments = { 0, 19, 56, 73, 40 };
        SetInstrument(instruments[comboInstrument.SelectedIn-
dex]);
    }

    // ----- Запис -----
    private void btnRecord_Click(object sender, EventArgs e)
    {
        recordedEvents.Clear();
        stopwatch.Reset();
        stopwatch.Start();
        isRecording = true;

        btnRecord.Enabled = false;
        btnStop.Enabled = true;
        btnPlay.Enabled = false;
        btnExport.Enabled = false;
    }

    private void btnStop_Click(object sender, EventArgs e)
    {
        if (isRecording)
        {
            stopwatch.Stop();
            isRecording = false;

            btnRecord.Enabled = true;
            btnStop.Enabled = false;
            btnPlay.Enabled = recordedEvents.Count > 0;
            btnExport.Enabled = recordedEvents.Count > 0;
        }
    }

    // ----- Възпроизвеждане на записа -----
    private async void btnPlay_Click(object sender, EventArgs e)
    {
        if (isPlayingBack)
        {
            playbackCancellation?.Cancel();
            return;
        }

        if (recordedEvents.Count == 0) return;
    }

```

```

        int program = GetSelectedInstrumentProgram();
        playbackCancellation = new CancellationTokenSource();
        var token = playbackCancellation.Token;

        isPlayingBack = true;
        btnPlay.Text = "Stop";
        btnRecord.Enabled = false;
        btnStop.Enabled = false;
        btnExport.Enabled = false;

        try
        {
            await Task.Run(() => PlaybackEvents(program, token),
token);
        }
        catch (OperationCanceledException)
        {
            // Потребителят е натиснал Stop
        }
        finally
        {
            isPlayingBack = false;
            btnPlay.Text = "Play";
            btnRecord.Enabled = !isRecording;
            btnStop.Enabled = isRecording;
            btnPlay.Enabled = recordedEvents.Count > 0;
            btnExport.Enabled = recordedEvents.Count > 0;

            // Изпращаме All Notes Off, за да спрем евентуално
залепнали звуци
            midiOut.Send(new ChannelMessage(ChannelCommand.Con-
troller, midiChannel, 123, 0));
            playbackCancellation?.Dispose();
            playbackCancellation = null;
        }
    }

    private void PlaybackEvents(int program, CancellationToken
token)
    {
        // Задаваме инструмента
        midiOut.Send(new ChannelMessage(ChannelCommand.Program-
Change, midiChannel, program, 0));
    }

```

```

        var sortedEvents = recordedEvents.OrderBy(e =>
e.TimeMs).ToList();
        Stopwatch sw = new Stopwatch();
        sw.Start();

        foreach (var ev in sortedEvents)
        {
            token.ThrowIfCancellationRequested();

            long targetMs = ev.TimeMs;
            while (sw.ElapsedMilliseconds < targetMs)
            {
                long remaining = targetMs - sw.ElapsedMillisecon-
onds;

                if (remaining > 15)
                    Thread.Sleep((int)(remaining - 5));
                else
                    Thread.SpinWait(100);
                token.ThrowIfCancellationRequested();
            }
            midiOut.Send(ev.Message);
        }

        sw.Stop();
        Thread.Sleep(300); // Малко изчакване за последните ноти
    }

    // ----- Експорт към WAV -----
    private async void btnExport_Click(object sender, EventArgs
e)
    {
        if (recordedEvents.Count == 0)
        {
            MessageBox.Show("Няма записани ноти.", "Информация",
MessageBoxButtons.OK, MessageBoxIcon.Information);
            return;
        }

        SaveFileDialog sfd = new SaveFileDialog();
        sfd.Filter = "WAV files (*.wav)|*.wav";
        if (sfd.ShowDialog() != DialogResult.OK)
            return;

        string outputFile = sfd.FileName;
        int program = GetSelectedInstrumentProgram();

```

```

        isExporting = true;
        btnExport.Enabled = false;
        btnRecord.Enabled = false;
        btnStop.Enabled = false;
        btnPlay.Enabled = false;

        WasapiLoopbackCapture capture = null;
        WaveFileWriter writer = null;

        try
        {
            capture = new WasapiLoopbackCapture();
            writer = new WaveFileWriter(outputFile, capture.Wave-
Format);

            capture.DataAvailable += (s, args) =>
            {
                writer.Write(args.Buffer, 0, args.BytesRecorded);
            };

            capture.RecordingStopped += (s, args) =>
            {
                writer?.Dispose();
                writer = null;
                capture?.Dispose();
            };

            capture.StartRecording();

            // Възпроизвеждаме записа, докато записваме от
звуковата карта
            await Task.Run(() => PlaybackEvents(program, Cancel-
lationToken.None));

            // Изчакваме малко повече, за да уловим остатъчния
звук
            await Task.Delay(2000);

            capture.StopRecording();

            MessageBox.Show("WAV файлът е създаден успешно!",
"Готово", MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
        catch (Exception ex)

```

```

        {
            MessageBox.Show("Грешка при експорт: " + ex.Message,
"Грешка", MessageBoxButtons.OK, MessageBoxIcon.Error);
            capture?.StopRecording();
        }
        finally
        {
            isExporting = false;
            btnRecord.Enabled = !isRecording;
            btnStop.Enabled = isRecording;
            btnPlay.Enabled = recordedEvents.Count > 0;
            btnExport.Enabled = recordedEvents.Count > 0;
        }
    }

    // Помощна функция за взимане на номера на избрания
инструмент
    private int GetSelectedInstrumentProgram()
    {
        int[] instruments = { 0, 19, 56, 73, 40 };
        return instruments[comboInstrument.SelectedIndex];
    }

    // ----- Почистване при затваряне -----
    private void MainForm_FormClosing(object sender, FormClos-
ingEventArgs e)
    {
        if (isRecording)
        {
            stopwatch.Stop();
            isRecording = false;
        }
        if (isPlayingBack)
        {
            playbackCancellation?.Cancel();
        }
    }
}

```

Фиг. 11 - пълен код на MainForm

## Приложение 2: пълен код на MainForm.Designer.cs

```
namespace PianoProject
{
    partial class MainForm
    {
        private System.ComponentModel.IContainer components = null;

        protected override void Dispose(bool disposing)
        {
            if (disposing != null && components != null) components.Dispose();
            base.Dispose(disposing);
        }

        #region Windows Form Designer generated code

        private void InitializeComponent()
        {
            this.components = new System.ComponentModel.Container();
            this.panelTop = new System.Windows.Forms.Panel();
            this.flowLayoutPanel1 = new System.Windows.Forms.FlowLayoutPanel();
            this.btnOctaveDown = new System.Windows.Forms.Button();
            this.btnOctaveUp = new System.Windows.Forms.Button();
            this.lblOctave = new System.Windows.Forms.Label();
            this.comboInstrument = new System.Windows.Forms.ComboBox();
            this.chkShowNotes = new System.Windows.Forms.CheckBox();
            this.btnRecord = new System.Windows.Forms.Button();
            this.btnStop = new System.Windows.Forms.Button();
            this.btnPlay = new System.Windows.Forms.Button();
            this.btnExport = new System.Windows.Forms.Button();
            this.panelSettings = new System.Windows.Forms.Panel();
            this.label1 = new System.Windows.Forms.Label();
            this.trackVolume = new System.Windows.Forms.TrackBar();
            this.panelKeyboard = new System.Windows.Forms.Panel();
            this.toolTip = new System.Windows.Forms.ToolTip(this.components);
            this.panelTop.SuspendLayout();
            this.flowLayoutPanel1.SuspendLayout();
            this.panelSettings.SuspendLayout();
        }

        #endregion
    }
}
```

```

        ((System.ComponentModel.ISupportInitialize)(this.track-
Volume)).BeginInit();
        this.SuspendLayout();
        //
        // panelTop
        //
        this.panelTop.BackColor = System.Drawing.Color.LightGray;
        this.panelTop.Controls.Add(this.flowLayoutPanel1);
        this.panelTop.Dock = System.Windows.Forms.DockStyle.Top;
        this.panelTop.Location = new System.Drawing.Point(0, 0);
        this.panelTop.Name = "panelTop";
        this.panelTop.Padding = new System.Windows.Forms.Pad-
ding(10);
        this.panelTop.Size = new System.Drawing.Size(933, 80);
        this.panelTop.TabIndex = 0;
        //
        // flowLayoutPanel1
        //
        this.flowLayoutPanel1.Controls.Add(this.btnOctaveDown);
        this.flowLayoutPanel1.Controls.Add(this.btnOctaveUp);
        this.flowLayoutPanel1.Controls.Add(this.lblOctave);
        this.flowLayoutPanel1.Controls.Add(this.comboInstrument);
        this.flowLayoutPanel1.Controls.Add(this.chkShowNotes);
        this.flowLayoutPanel1.Controls.Add(this.btnRecord);
        this.flowLayoutPanel1.Controls.Add(this.btnStop);
        this.flowLayoutPanel1.Controls.Add(this.btnPlay);
        this.flowLayoutPanel1.Controls.Add(this.btnExport);
        this.flowLayoutPanel1.Dock = System.Windows.Forms.Dock-
Style.Fill;
        this.flowLayoutPanel1.FlowDirection = System.Win-
dows.Forms.FlowDirection.LeftToRight;
        this.flowLayoutPanel1.Location = new System.Draw-
ing.Point(10, 10);
        this.flowLayoutPanel1.Name = "flowLayoutPanel1";
        this.flowLayoutPanel1.Padding = new System.Win-
dows.Forms.Padding(0);
        this.flowLayoutPanel1.Size = new System.Drawing.Size(913,
60);
        this.flowLayoutPanel1.TabIndex = 0;
        this.flowLayoutPanel1.WrapContents = false;
        //
        // btnOctaveDown
        //
        this.btnOctaveDown.FlatStyle = System.Win-
dows.Forms.FlatStyle.Flat;

```



```

        this.btnOctaveDown.Font = new System.Drawing.Font("Segoe
UI", 12F, System.Drawing.FontStyle.Bold);
        this.btnOctaveDown.Location = new System.Drawing.Point(3,
12);
        this.btnOctaveDown.Margin = new System.Windows.Forms.Pad-
ding(3, 12, 0, 3);
        this.btnOctaveDown.Name = "btnOctaveDown";
        this.btnOctaveDown.Size = new System.Drawing.Size(50,
35);
        this.btnOctaveDown.TabIndex = 0;
        this.btnOctaveDown.Text = "◀";
        this.toolTip.SetToolTip(this.btnOctaveDown, "Lower oc-
tave");
        this.btnOctaveDown.UseVisualStyleBackColor = true;
        //
        // btnOctaveUp
        //
        this.btnOctaveUp.FlatStyle = System.Win-
dows.Forms.FlatStyle.Flat;
        this.btnOctaveUp.Font = new System.Drawing.Font("Segoe
UI", 12F, System.Drawing.FontStyle.Bold);
        this.btnOctaveUp.Location = new System.Drawing.Point(53,
12);
        this.btnOctaveUp.Margin = new System.Windows.Forms.Pad-
ding(0, 12, 10, 3);
        this.btnOctaveUp.Name = "btnOctaveUp";
        this.btnOctaveUp.Size = new System.Drawing.Size(50, 35);
        this.btnOctaveUp.TabIndex = 1;
        this.btnOctaveUp.Text = "▶";
        this.toolTip.SetToolTip(this.btnOctaveUp, "Higher oc-
tave");
        this.btnOctaveUp.UseVisualStyleBackColor = true;
        //
        // lblOctave
        //
        this.lblOctave.AutoSize = true;
        this.lblOctave.Font = new System.Drawing.Font("Segoe UI",
14F, System.Drawing.FontStyle.Bold);
        this.lblOctave.Location = new System.Drawing.Point(116,
17);
        this.lblOctave.Margin = new System.Windows.Forms.Pad-
ding(3, 17, 10, 3);
        this.lblOctave.Name = "lblOctave";
        this.lblOctave.Size = new System.Drawing.Size(35, 25);
        this.lblOctave.TabIndex = 2;

```

```

        this.lblOctave.Text = "C5";
        this.toolTip.SetToolTip(this.lblOctave, "Current oc-
tave");
        //
        // comboInstrument
        //
        this.comboInstrument.DropDownStyle = System.Win-
dows.Forms.ComboBoxStyle.DropDownList;
        this.comboInstrument.Font = new System.Draw-
ing.Font("Segoe UI", 10F);
        this.comboInstrument.FormattingEnabled = true;
        this.comboInstrument.Items.AddRange(new object[] {
            "Piano",
            "Organ",
            "Trumpet",
            "Flute",
            "Violin"});
        this.comboInstrument.Location = new System.Draw-
ing.Point(164, 17);
        this.comboInstrument.Margin = new System.Win-
dows.Forms.Padding(3, 17, 10, 3);
        this.comboInstrument.Name = "comboInstrument";
        this.comboInstrument.Size = new System.Drawing.Size(140,
25);
        this.comboInstrument.TabIndex = 3;
        this.toolTip.SetToolTip(this.comboInstrument, "Select in-
strument");
        //
        // chkShowNotes
        //
        this.chkShowNotes.AutoSize = true;
        this.chkShowNotes.Checked = true;
        this.chkShowNotes.CheckState = System.Win-
dows.Forms.CheckState.Checked;
        this.chkShowNotes.Font = new System.Drawing.Font("Segoe
UI", 10F);
        this.chkShowNotes.Location = new System.Draw-
ing.Point(317, 17);
        this.chkShowNotes.Margin = new System.Windows.Forms.Pad-
ding(3, 17, 10, 3);
        this.chkShowNotes.Name = "chkShowNotes";
        this.chkShowNotes.Size = new System.Drawing.Size(99, 23);
        this.chkShowNotes.TabIndex = 4;
        this.chkShowNotes.Text = "Show notes";

```

```

        this.toolTip.SetToolTip(this.chkShowNotes, "Show note
names on keys");
        this.chkShowNotes.UseVisualStyleBackColor = true;
        //
        // btnRecord
        //
        this.btnRecord.FlatStyle = System.Win-
dows.Forms.FlatStyle.Flat;
        this.btnRecord.Font = new System.Drawing.Font("Segoe UI",
10F, System.Drawing.FontStyle.Bold);
        this.btnRecord.Location = new System.Drawing.Point(429,
12);
        this.btnRecord.Margin = new System.Windows.Forms.Pad-
ding(3, 12, 5, 3);
        this.btnRecord.Name = "btnRecord";
        this.btnRecord.Size = new System.Drawing.Size(70, 35);
        this.btnRecord.TabIndex = 5;
        this.btnRecord.Text = "Record";
        this.toolTip.SetToolTip(this.btnRecord, "Start record-
ing");
        this.btnRecord.UseVisualStyleBackColor = true;
        //
        // btnStop
        //
        this.btnStop.Enabled = false;
        this.btnStop.FlatStyle = System.Win-
dows.Forms.FlatStyle.Flat;
        this.btnStop.Font = new System.Drawing.Font("Segoe UI",
10F, System.Drawing.FontStyle.Bold);
        this.btnStop.Location = new System.Drawing.Point(504,
12);
        this.btnStop.Margin = new System.Windows.Forms.Padding(0,
12, 5, 3);
        this.btnStop.Name = "btnStop";
        this.btnStop.Size = new System.Drawing.Size(70, 35);
        this.btnStop.TabIndex = 6;
        this.btnStop.Text = "Stop";
        this.toolTip.SetToolTip(this.btnStop, "Stop recording");
        this.btnStop.UseVisualStyleBackColor = true;
        //
        // btnPlay
        //
        this.btnPlay.Enabled = false;
        this.btnPlay.FlatStyle = System.Win-
dows.Forms.FlatStyle.Flat;

```

```

        this.btnPlay.Font = new System.Drawing.Font("Segoe UI",
10F, System.Drawing.FontStyle.Bold);
        this.btnPlay.Location = new System.Drawing.Point(579,
12);
        this.btnPlay.Margin = new System.Windows.Forms.Padding(0,
12, 5, 3);
        this.btnPlay.Name = "btnPlay";
        this.btnPlay.Size = new System.Drawing.Size(70, 35);
        this.btnPlay.TabIndex = 8;
        this.btnPlay.Text = "Play";
        this.toolTip.SetToolTip(this.btnPlay, "Play recorded");
        this.btnPlay.UseVisualStyleBackColor = true;
        //
        // btnExport
        //
        this.btnExport.Enabled = false;
        this.btnExport.FlatStyle = System.Win-
dows.Forms.FlatStyle.Flat;
        this.btnExport.Font = new System.Drawing.Font("Segoe UI",
10F, System.Drawing.FontStyle.Bold);
        this.btnExport.Location = new System.Drawing.Point(654,
12);
        this.btnExport.Margin = new System.Windows.Forms.Pad-
ding(0, 12, 3, 3);
        this.btnExport.Name = "btnExport";
        this.btnExport.Size = new System.Drawing.Size(90, 35);
        this.btnExport.TabIndex = 7;
        this.btnExport.Text = "Export WAV";
        this.toolTip.SetToolTip(this.btnExport, "Save as WAV");
        this.btnExport.UseVisualStyleBackColor = true;
        //
        // panelSettings
        //
        this.panelSettings.BackColor = System.Draw-
ing.Color.White;
        this.panelSettings.BorderStyle = System.Win-
dows.Forms.BorderStyle.FixedSingle;
        this.panelSettings.Controls.Add(this.label1);
        this.panelSettings.Controls.Add(this.trackVolume);
        this.panelSettings.Dock = System.Windows.Forms.Dock-
Style.Top;
        this.panelSettings.Location = new System.Drawing.Point(0,
80);
        this.panelSettings.Name = "panelSettings";

```

```

        this.panelSettings.Padding = new System.Windows.Forms.Padding(15);
        this.panelSettings.Size = new System.Drawing.Size(933, 80);

        this.panelSettings.TabIndex = 1;
        //
        // label1
        //
        this.label1.AutoSize = true;
        this.label1.Font = new System.Drawing.Font("Segoe UI", 10F);

        this.label1.Location = new System.Drawing.Point(15, 15);
        this.label1.Margin = new System.Windows.Forms.Padding(3, 3, 0);

        this.label1.Name = "label1";
        this.label1.Size = new System.Drawing.Size(55, 19);
        this.label1.TabIndex = 0;
        this.label1.Text = "Volume";
        this.toolTip.SetToolTip(this.label1, "Volume level");
        //
        // trackVolume
        //
        this.trackVolume.LargeChange = 10;
        this.trackVolume.Location = new System.Drawing.Point(15, 37);

        this.trackVolume.Margin = new System.Windows.Forms.Padding(3, 0, 3, 3);
        this.trackVolume.Maximum = 100;
        this.trackVolume.Name = "trackVolume";
        this.trackVolume.Size = new System.Drawing.Size(350, 45);
        this.trackVolume.SmallChange = 10;
        this.trackVolume.TabIndex = 1;
        this.trackVolume.TickFrequency = 10;
        this.toolTip.SetToolTip(this.trackVolume, "Adjust volume");

        this.trackVolume.Value = 100;
        //
        // panelKeyboard
        //
        this.panelKeyboard.BackColor = System.Drawing.Color.DarkGray;
        this.panelKeyboard.Dock = System.Windows.Forms.DockStyle.Fill;
        this.panelKeyboard.Location = new System.Drawing.Point(0, 160);

```

```

        this.panelKeyboard.Name = "panelKeyboard";
        this.panelKeyboard.Size = new System.Drawing.Size(933,
411);
        this.panelKeyboard.TabIndex = 2;
        //
        // toolTip
        //
        this.toolTip.AutomaticDelay = 300;
        this.toolTip.ShowAlways = true;
        //
        // MainForm
        //
        this.AutoScaleDimensions = new System.Drawing.SizeF(7F,
15F);
        this.AutoScaleMode = System.Windows.Forms.AutoScale-
Mode.Font;
        this.BackColor = System.Drawing.Color.WhiteSmoke;
        this.ClientSize = new System.Drawing.Size(933, 571);
        this.Controls.Add(this.panelKeyboard);
        this.Controls.Add(this.panelSettings);
        this.Controls.Add(this.panelTop);
        this.Font = new System.Drawing.Font("Segoe UI", 9F);
        this.KeyPreview = true;
        this.Name = "MainForm";
        this.StartPosition = System.Windows.Forms.FormStartPosi-
tion.CenterScreen;
        this.Text = "Piano";
        this.panelTop.ResumeLayout(false);
        this.flowLayoutPanel1.ResumeLayout(false);
        this.flowLayoutPanel1.PerformLayout();
        this.panelSettings.ResumeLayout(false);
        this.panelSettings.PerformLayout();
        ((System.ComponentModel.ISupportInitialize)(this.track-
Volume)).EndInit();
        this.ResumeLayout(false);
    }

    #endregion

    private System.Windows.Forms.Panel panelTop;
    private System.Windows.Forms.FlowLayoutPanel flowLayout-
Panel1;
    private System.Windows.Forms.Button btnOctaveDown;
    private System.Windows.Forms.Button btnOctaveUp;

```

```
private System.Windows.Forms.Label lblOctave;  
private System.Windows.Forms.ComboBox comboInstrument;  
private System.Windows.Forms.CheckBox chkShowNotes;  
private System.Windows.Forms.Button btnRecord;  
private System.Windows.Forms.Button btnStop;  
private System.Windows.Forms.Button btnPlay;  
private System.Windows.Forms.Button btnExport;  
private System.Windows.Forms.Panel panelSettings;  
private System.Windows.Forms.TrackBar trackVolume;  
private System.Windows.Forms.Label label1;  
private System.Windows.Forms.Panel panelKeyboard;  
private System.Windows.Forms.ToolTip toolTip;  
}  
}
```

*Фиг. 12 - пълен код на MainForm.Designer.cs*

## ИЗПОЛЗВАНА ЛИТЕРАТУРА:

1. Albahari, J., & Albahari, B. (2017). *C# 7.0 in a Nutshell*. O'Reilly Media.  
<https://www.oreilly.com/library/view/c-7-0/9781491987650/>
2. General MIDI Association. (n.d.). *General MIDI Level 1 Specification*.  
<https://midi.org/general-midi>
3. Heath, M. (n.d.). *NAudio [GitHub repository]*. GitHub. <https://github.com/naudio/NAudio>
4. Microsoft Docs. (2025). *.NET API Browser (Stopwatch, Thread.Sleep, Thread.SpinWait)*. Microsoft Learn. <https://learn.microsoft.com/en-us/dotnet/api/>
5. Microsoft Docs. (2025). *Asynchronous programming with async and await*. Microsoft Learn. <https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/concepts/async/>
6. Microsoft Docs. (2025). *C# Programming Guide*. Microsoft Learn. <https://learn.microsoft.com/en-us/dotnet/csharp/>
7. Microsoft Docs. (2025). *Managed Threading in .NET*. Microsoft Learn. <https://learn.microsoft.com/en-us/dotnet/standard/threading/>
8. MIDI Manufacturers Association. (1996). *The Complete MIDI 1.0 Detailed Specification*.  
<https://midi.org/midi-1-0-detailed-specification>
9. NAudio Documentation. (n.d.). *WasapiLoopbackCapture*. GitHub Wiki.  
<https://github.com/naudio/NAudio/blob/master/Docs/WasapiLoopbackCapture.md>
10. Russ, M. (2019). *Sound Synthesis and Sampling* (4th ed.). Focal Press.  
<https://www.routledge.com/Sound-Synthesis-and-Sampling/Russ/p/book/9781138360051>
11. Sanford, L. (n.d.). *Sanford.Multimedia.Midi [GitHub repository]*. GitHub.  
<https://github.com/tebjan/Sanford.Multimedia.Midi>
12. W3Schools. (2024). *C# Tutorial*. Refsnes Data. <https://www.w3schools.com/CS/index.php>